Python Application Deployment on AWS Cloud Platform

A. Provision and Deploy PostgreSQL Database on Amazon RDS [1 hr]:
   1. Go to Amazon RDS
   2. Select Full Configuration
   3. PostgreSQL (Community Edition) -> Sandbox Template
   4. Instance Type: db.t3.micro
   5. Region: us-east-1
   6. Provide Identifier: ecommerce_database, Master Username & Password
   7. Keep default role, AZ, VPC and subnet, SecGroup: default; rds-ec2-1
   8. Size: 20GB, Auto Scaled to 100 GB
   9. Port 5432
   10. Auto backup cycle, retention-1 day, backup at 18:00 hrs daily

B. Provision EC2 backend application server [2 hrs]:

1. Go to AWS EC2
2. Select Launch Instance option
3. Identifier: ecommerce_backend
4. AMI: Ubuntu: 26.04 LTS, Instance Type: t2.medium
5. Storage 25GB, SecGroup: Launch Wizard-1; ec2-rds-1, VPC: default, Key: mykey
6. Connect to EC2 terminal and run commands:
    a. sudo apt update && sudo apt upgrade -y
    b. sudo apt install python3 python-is-python3 python3-pip python3-fastapi apache2 postgresql zip unzip virtualenv -y
    c. To update apache2 config follow steps ahead:
    d. sudo a2enmod proxy
    e. sudo a2enmod proxy_http
    f. sudo nano /etc/apache2/sites-available/000-default.conf
    g. ProxyPreserveHost On
    h. ProxyPass / http://127.0.0.1:8000/
    i. ProxyPassReverse / http://127.0.0.1:8000/

glish (U.S.) Text Predictions: Off Editor Suggestions: Showing 100% Fit Give Fe

C. Create S3 Buckets for sourcing and deployment of Front End [30 mins]

1. Create General Purpose Buckets
    a. sourcecode-xy-1234 [xy is initials; 1234 any numbers]
    b. frontend-http-xy-1234
    c. frontend-https-xy-1234
2. Block public access: OFF
3. Versioning: Enabled

Reconnecting...

C. Create S3 Buckets for sourcing and deployment of Front End [30 mins]

1. Create General Purpose Buckets
    a. sourcecode-xy-1234 [xy is initials; 1234 any numbers]
    b. frontend-http-xy-1234
    c. frontend-https-xy-1234
2. Block public access: OFF
3. Versioning: Enabled
4. Setup permission for frontend buckets
5. Setup static website hosting for frontend buckets

D. Create credential on AWS Secret Manager [30 mins]:

1. Go to Secrets Manager
2. Create new credentials; secret name: ecommerce-db-secrets
3. Select RDS, Add DB master username and password
4. Select DB instance identifier for selection of DB endpoint URL

Reconnecting...
Search

Search for tools, help, and more (Alt + Q)

Layout References Review View Help

Aptos Bold 18 B I U Normal

D. Create credential on AWS Secret Manager [30 mins]:

1. **Go to Secrets Manager**
2. **Create new credentials; secret name: ecommerce-db-secrets**
3. **Select RDS, Add DB master username and password**
4. **Select DB instance identifier for selection of DB endpoint URL**
5. **Click create. Refer SDK connection for Python code**

E. Build Frontend for Deployment to S3 & upload Backend [1 hr]:

1. **Edit .env.production or .env file for public IP of your EC2**

   http://<public-IP>

Reconnecting...

Search

2. Create a ZIP file for your frontend directory
3. Edit .env file for https URL
4. Create another ZIP of your frontend directory with suffix as -https
5. Upload both ZIP files to s3://sourcecode-xy-1234 bucket
6. Create ZIP file for backend and upload to sourcecode-xy-1234 bucket

F. Create CodeBuild project to build http and https frontend codes:
   1. Create project: frontend-build-http
   2. Select source bucket: s3://sourcecode-xy-1234/frontend-http.zip
   3. Edit buildspec file for respective source and destination bucket
   4. Create a similar project for https frontend code with respective destination bucket
   5. Run Build for http and https projects, if needed, attach S3FullAccess permission to CodeBuild Role.

G. Connect Backend to Database:

1. Download backend ZIP file from S3 inside application folder
2. Unzip the ZIP file
3. Install requirements and create, if required, .env file
4. Update DB connection python code to use Boto3 and SDK for python to get credentials from secret manager using secret name, region and other details.
5. Update main.py file for running on port 8000 and 127.0.0.1 as host
6. Test RUN Backend with DB connected on Uvicorn
7. Manually test API endpoints using Swagger UI

1. Create Lambda function for regular health check after every 1 minute
2. Create (optional) Lambda function as triggered from S3 update for Frontend or Backend
3. Enable and install open-sll in EC2 Backend server, with apache2 configuration
4. Configure S3 bucket and CloudFront for Edge Distribution